

Zdroje:

https://www.w3schools.com/js/js_ajax_intro.asp

<http://api.jquery.com/jquery.ajax/>

Obsah

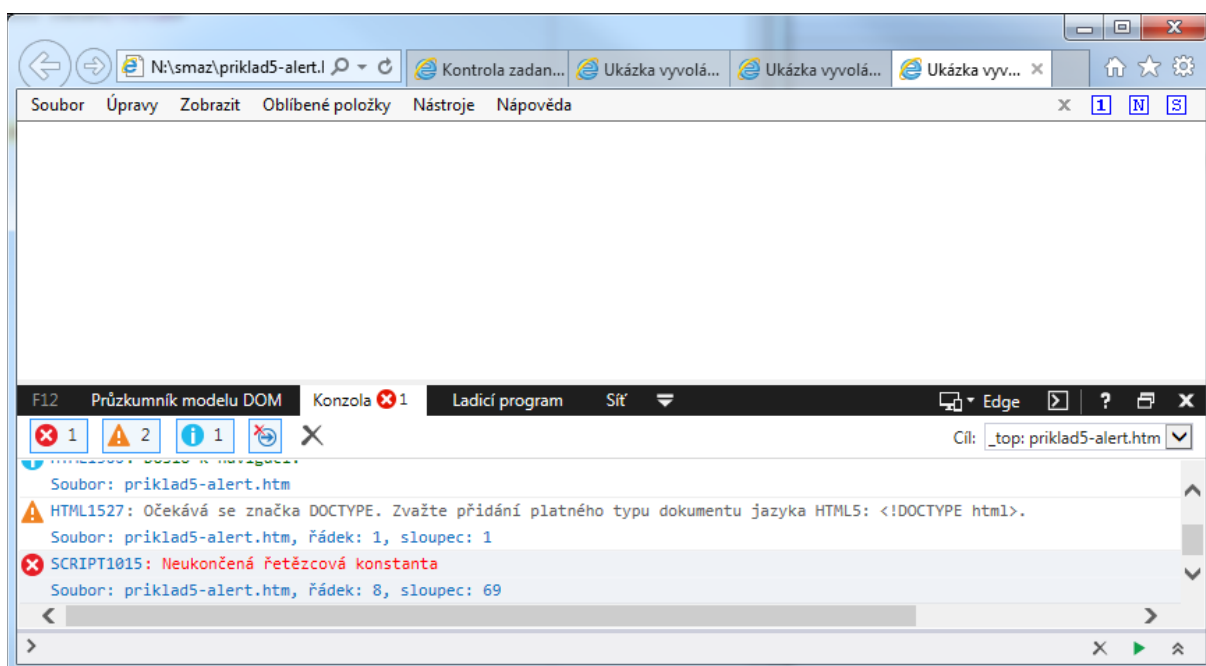
1	Hledání chyb v JavaScriptu.....	2
2	Zkoušečka jQuery.....	5
3	Ajax.....	6
3.1	Stručná historie.....	6
3.2	Princip AJAXu.....	6
3.3	Pro posílání požadavku na server se používá XHR –XMLHttpRequest().....	6
3.4	Použití AJAXu.....	6
3.5	Jaká data AJAX posílá.....	7
3.5.1	CSS.....	7
3.5.2	Prostý text.....	7
3.5.3	HTML.....	7
3.5.4	Vykonatelný JavaScript.....	7
3.5.5	XML.....	8
3.6	JSON.....	8
4	XMLHttpRequest a jak to vlastně pracuje.....	10
4.1.1	Open().....	10
4.1.2	ODESÍLÁNÍ DAT - metoty.....	11
4.1.3	Send().....	11
4.1.1	Zpracování odpovědi na XMLHttpRequest.....	11
4.1.4	RUČNÍ ODESÍLÁNÍ DAT.....	12
5	Ukázka jednoduchého požadavku.....	12
6	Přenos textu ze serveru.....	12
7	Čtení souboru XML pomocí asynchronního požadavku XHR.....	17
1.1	XML.....	17
8	Změna obsahu dokumentu HTML pomocí innerHTML.....	21
9	Ajax - Dynamické vložení tabulky z externího dokumentu (HTML).....	21
10	Vložení textu vygenerovaného serverem v PHP.....	23
11	PHP s parametrem.....	24
12	Jednoduchý kalkulátor.....	27

1.2	Neprve klasika.....	27
13	Interaktivní ajaxový kalkulátor bez potvrzování.....	28
14	Ladění doby trvání běhu kódu Ajax aplikace.....	31
15	Identifikace prohlížeče v JavaScriptu.....	32

1 Hledání chyb v JavaScriptu

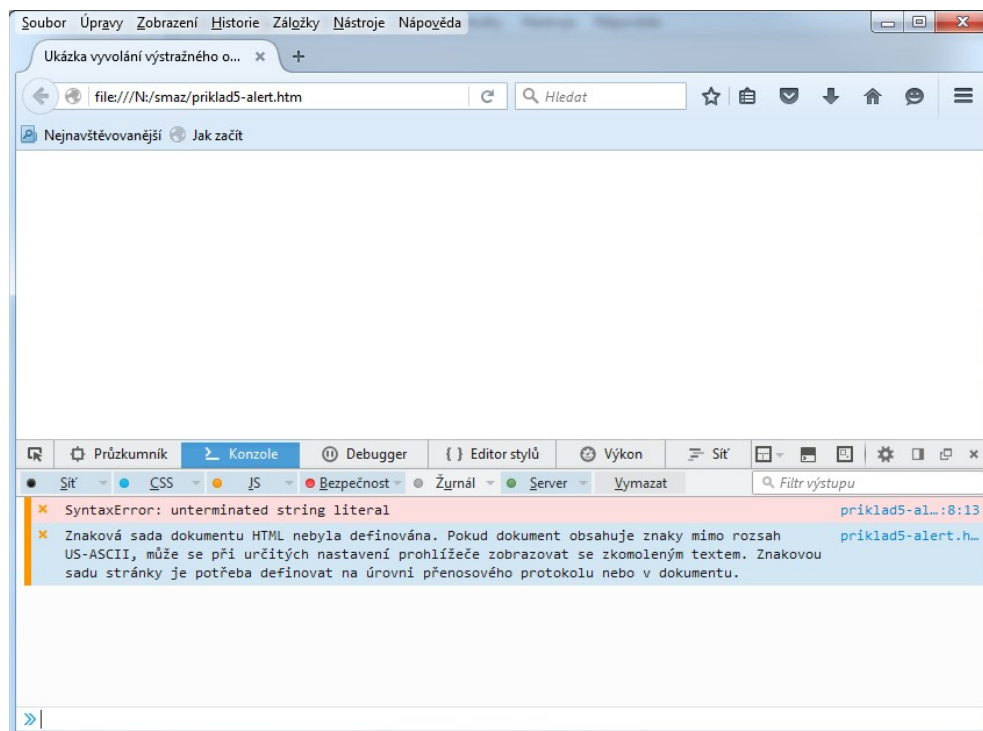
IE

Spustit Nástroje – Vývojářské nástroje.

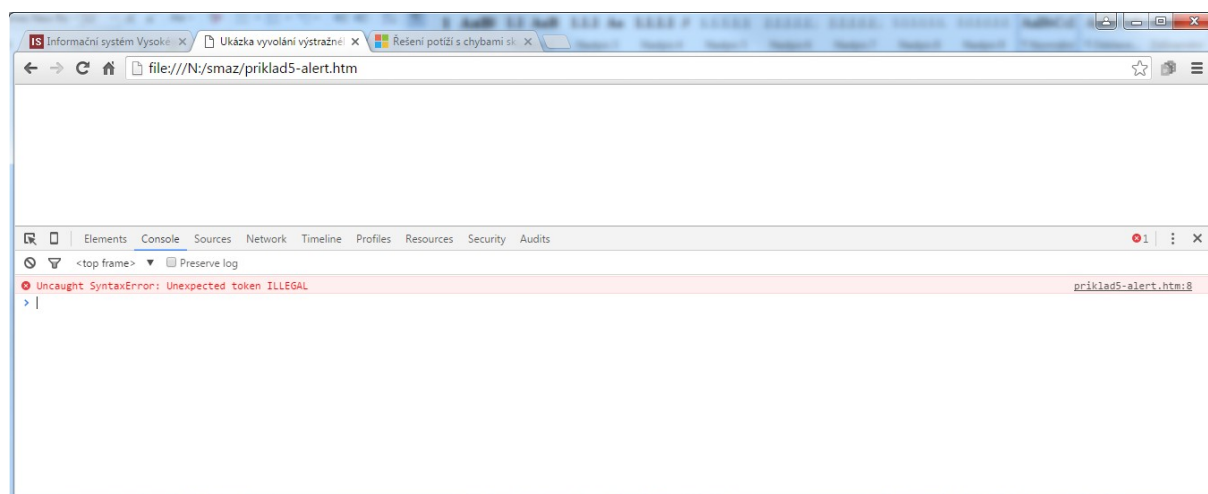
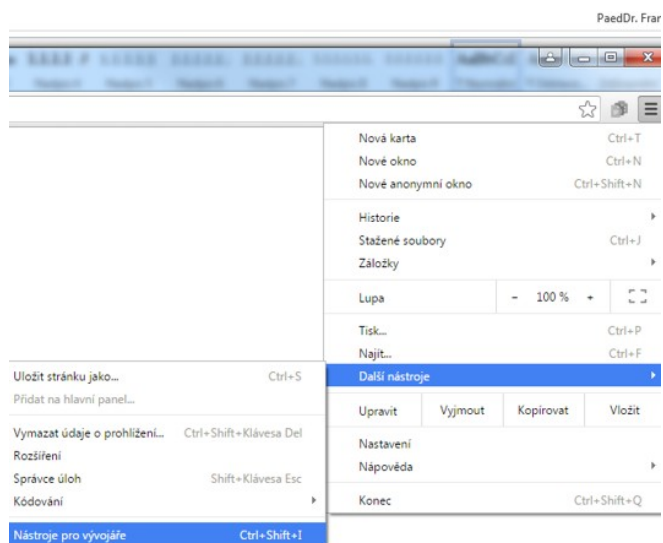


Mozilla Firefox

Vývojář – Nástroje vývojáře



V google chrome Další nástroje – nástroje pro vývojáře.



2 Zkoušečka jQuery

<http://jquery.jslab.net/zkousecka/>

Pracuje, je nutné na prvek kliknout (2024).

jQuery zkoušečka [Nápověda](#)

Kód [reset]

HTML kód:

```
<div id="ahoj">Ahoj!</div>
```

JavaScript:

```
$(document).ready(function(){  
});
```

Verze jQuery: jQuery 2.0.3

Spustit!

Výsledek

zatím nic

Lze zkusit:

```
$(document).ready(function(){  
    $("div").click(function(){  
        $(this).hide();  
    });  
});
```

```
$(document).ready(function(){  
    $("div").click(function(){  
        $(this).css("background-color", "yellow");  
    });  
});
```

3 Ajax

- Dynamické změny obsahu stránek
- AJAX = Asynchronous JavaScript And XML
- XHR = XMLHttpRequest - metoda pro posílání požadavků
- Ajax je sada technik a nástrojů, které umožňují dynamické změny obsahu stránky, interakci s webem a načítání částí obsahu ze serveru bez znovunačtení stránek.

3.1 Stručná historie

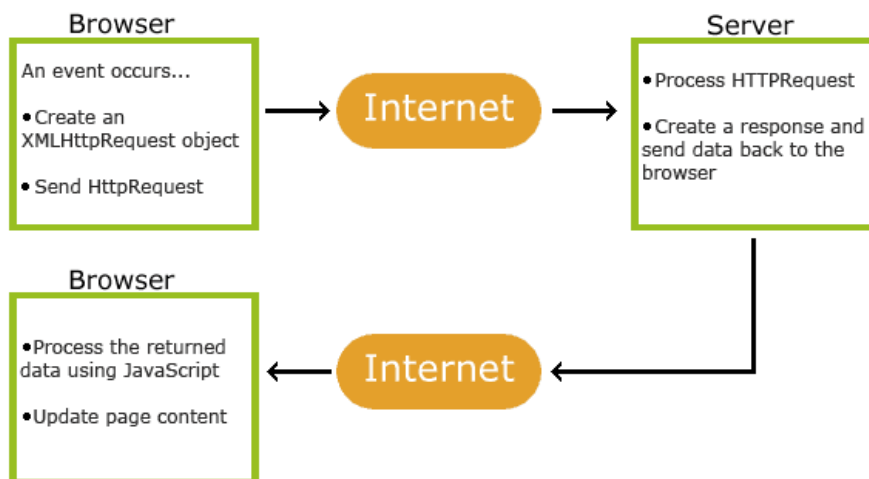
- Předávání dat skrytým rámcem (iframe)(používá se dodnes –"AJAX pro chudé")
- Od roku 2005 se tento přístup k psaní aplikací pojmenovává jako AJAX (autor pojmu – Jesse James Garrett)

3.2 Princip AJAXu

- Uživatel nějak interaguje se stránkou (klikne, ...) a webová stránka by v tu chvíli měla změnit zobrazená data. Jak?
 1. Data má připravená a schovaná, jen je zobrazí
 2. Odešle na server požadavek, a ten vrátí novou stránku s novými daty
 3. Odešle na server požadavek, a server vrátí jen požadovaná data. Stránka je musí pak zpracovat a zobrazit. AJAX!

3.3 Pro posílání požadavku na server se používá XHR –XMLHttpRequest().

- XHR umí poslat asynchronní požadavek na určitou adresu
- Asynchronní: program pracuje normálně dál, a když se něco zajímavého stane, systém zavolá určenou funkci.



3.4 Použití AJAXu

- Aktualizovat webové stránky bez reloadu stránky
- Žádost o data ze serveru - po načtení stránky
- Přijímat data ze serveru - po načtení stránky
- Odesílat data na server - v pozadí

4 Jaká data AJAX posílá

4.1 CSS

4.2 Prostý text

Server pošle jen prostý text – zpracujete pomocí **xhr.responseText()**;

Výhoda:

extrémně jednoduchý

Nevýhoda:

nepřenesení složitější data, jen prostý řetězec

4.3 HTML

Server pošle část HTML, kterou si vložíte do stránky pomocí **innerHTML** – zpracujete pomocí **xhr.responseText()**;

```
function onSuccess(xhr) {  
  var div = document.getElementById('response');  
  div.innerHTML = xhr.responseText;  
}  
<div id="response">  
</div>
```

4.4 Vykonatelný JavaScript

Odpověď od serveru je JS kód, který se na klientu spustí.

```
function onSuccess(o) {eval(o.responseText);}
```

Výhody:

jednoduché zpracování

server může poslat doslova "cokoli" a vyvolat jakoukoli reakci prohlížeče, není omezen designem aplikace

4.5 XML

Server může poslat strukturovaná data jako XML; prohlížeče mají zabudované XML parsery, takže zpracování není složité:

```
<person>
<name>Pepa</name>
<age>18</age>
</person>
```

```
function onSuccess(xhr) {
var dom = xhr.responseXML;
showPerson(
dom.getElementsByTagName('name')[0].data,
parseInt(dom.getElementsByTagName('age')[0].data, 10)
});
}
```

Výhody: Oprášíte si znalosti DOMu

xhr.responseXML() parsuje za nás

Mnohé služby jsou založené na XML

Nevýhody: Složitá práce s DOMem

4.6 JSON

(JavaScript Object Notation.)

Příklad

```
var myObj = { "name": "John", "age": 31, "city": "New York" };
```

John New York 31

Mnohem jednodušší, stručnější formát. Dokáže popsat běžné datové struktury (seznam, pole) a je nativní součástí Javascriptu. K vyhodnocení stačí prosté eval().

POZOR!!!!

eval() is evil()! Nepoužívejte přímo eval(), protože tím spustíte kód! Pokud pracujete s JSON, použijte raději specializované parsery na data.

json_encode Vrátí řetězec obsahující reprezentaci JSON z hodnoty.

```
<?php
```

```
$ret = array();
$ret['stav'] = 0;
$ret['text'] = 'Mysis se prihlasit';
```



```

echo json_encode($ret);
echo "<br><br><br><br><br>";

$json_string = '{"stav":0,"text":"Mus\u00edd\u0161 se
p\u0159ihl\u00e1sit","citat":"\u00e9pe pozd\u011b, ne\u017e
nikdy.\u0022}';
$json = json_decode($json_string, true);

// array (
// 'stav' => 0,
// 'text' => 'Musíš se přihlásit',
// 'citat' => '"Lépe pozde, než nikdy."',
// )
var_export($json);

echo $json['citat']; // "Lépe pozde, než nikdy."
?>

```

<http://195.113.207.163/~smrcka/wt2/json/json.php>
 {"stav":0,"text":"Musiš se přihlásit"}

```

array ( 'stav' => 0, 'text' => 'Musíš se přihlásit', 'citat' => '"Lépe pozdě, než nikdy."', )"Lépe pozdě, než nikdy."

```

5 XMLHttpRequest a jak to vlastně pracuje

XMLHttpRequest (XHR)

- Je rozhraní umožňující webovým aplikacím komunikaci mezi serverem a klientem prostřednictvím protokolu HTTP.
- Nejčastěji je tento komunikační prostředek používán v interaktivních webových aplikacích založených na technologii AJAX umožňující změnu části obsahu stránky zobrazené v prohlížeči klienta bez nutnosti jejího opětovného kompletního načtení ze serveru.

Problém

- Objekt **XMLHttpRequest** se v prohlížeči Internet Explorer interpretuje jako **ActiveX**
- V ostatních prohlížečích je implementovaná třída **XMLHttpRequest** jako nativní objekt **JavaScriptu**.
- Tedy zjistíme, zda je možné vytvořit JavaScriptový objekt **XMLHttpRequest**. **Tuto metodu podporují Mozilla Firefox, Safari a Opera.**
- Pro IE tvoříme objekty ActiveX.
- Do verze IE 5.0 musíme vytvořit objekt Microsoft.XMLHTTP.
- V novějších verzích IE můžeme použít objekt Msxml2.XMLHTTP.
- Objekt XMLHttpRequest zapouzdřuje několik metod a atributů

Metoda/vlastnost	Popis
abort()	Ukončí aktuální požadavek
new XMLHttpRequest()	Vytvoří nový objekt XMLHttpRequest()
getAllResponseHeaders()	Vrátí hlavičky odpovědi jako řetězec
getResponseHeader("headerLabel")	Vrátí jednu hlavičku odpovědi jako řetězec
open(method, url, async, user, psw)	Inicializuje parametry požadavku method: typ požadavku GET or POST url: adresa souboru, Pokud URL směřuje do jiné domény, vyvolá se výjimka async: true (asynchronous) or false (synchronous) user: volitelné uživatelské jméno psw: heslo jméno a heslo jsou nepovinné parametry pro autentifikace a autorizaci
send()	pošle požadavek serveru Použití pro GET požadavek
send(string)	pošle požadavek serveru Použití pro POST požadavek
setRequestHeader()	Nastaví dvojici label/value hlavičky požadavku
onreadystatechange	Používá se pro funkci zpětného volání, která ovládá

Metoda/vlastnost	Popis
	změny stavu požadavku
readyState	Vrátí stav požadavku: 0 = neinicializovaný 1 = načítání 2 = načten 3 = interaktivní 4 = dokončeno
responseText	Vrátí odpověď serveru jako řetězec
responseXML	Vrátí odpověď serveru jako dokument XML
Status	Vrátí stavový požadavek
statusText	Vrátí zprávu o stavu požadavku

Syntaxe pro vytvoření `XMLHttpRequest` objektu:

```
variable = new XMLHttpRequest();
```

Příklad

```
var xhttp = new XMLHttpRequest();
```

```
xhttp.open("GET", "Obsah.txt", true);
```

5.1.1 ODESÍLÁNÍ DAT - metody

Protokol HTTP podporuje několik metod pro odesílání požadavků

- GET – nejběžnější; čtení dat a odeslání krátkých dat neměnicích stav backendu
- POST – odeslání dat, změna stavu backendu

5.1.2 Send()

provede přenos údajů v těle požadavků.

syntaxe

```
void send(data)
```

```
xhttp.open("GET", "ajax_info.txt", true);
```

```
xhttp.send();
```

1.1.1 Zpracování odpovědi na XMLHttpRequest

Po přijetí požadavku typu XMLHttpRequest přijme serveru od klienta data, která nejčastěji obsahují klientem zadané údaje.

5.1.3 ODESÍLÁNÍ DAT s parametrem

pro GET

```
xhr.open("GET", "skript.php?param1=hodnota&param2=hodnota");  
xhr.send();
```

pro POST

```
xhr.open("POST", "skript.php");  
xhr.send("param1=hodnota&param2=hodnota");
```

6 Přenos textu ze serveru

Text je v souboru

Aby vše fungovalo, musíme se dozvědět, kdy byla na náš server vygenerována odpověď.

Proto definujeme funkci pro obsluhu události stavu:

```
xHttp.onreadystatechange = obsluhaStateChange;
```

Pomocí metody open zpracujeme odpověď ze serveru:

```
xHttp.open("GET", "Obsah.txt", true);
```

První parametr určuje metodu požadavku http.

V případě, když od serveru požadujeme údaje, použijeme metodu GET,

V případě, když serveru jen posíláme informace, použijeme metodu POST.

.

GET – pro přenos údajů do 0,5 KB

POST - pro přenos údajů nad 0,5 KB

```
function aktivujRequest()  
{  
    xHttp = VytvorXMLHttp();  
    xHttp.onreadystatechange = obsluhaStateChange;  
    xHttp.open("GET", "Obsah.txt", true);  
    xHttp.send(null);  
}
```

Pro nás důležitý 4, což znamená, že odpověď ze serveru přišla kompletní, a my můžeme pokračovat v jejím zpracování.

Kromě stavu musíme při zpracování odpovědi na náš požadavek zkontrolovat i stav odpovědi serveru. Důležité je, aby stavový kód serveru http měl hodnotu 200 (OK)

```
function obsluhaStateChange()  
{
```

```

    if(xHttp.readyState == 4 && xHttp.status == 200)
    {
        alert("Odpověď serveru: " + xHttp.responseText);
    }
}

```

Poznámka

200 Zdroj byl nalezen a vše je OK

Pro zpracování nestrukturovaného textu využíváme metodu `xmlHttp.responseText`.

V reálných aplikacích, kdy na náš požadavek není statický soubor, ale obsah generovaný dynamickým serverem, je potřebné poslat hlavičku **Cache-Control: no-cache**, aby se zamezilo ukládání souboru do vyrovnávací paměti (cache) prohlížeče.

Načtení obsahu souboru txt do stránky pomocí AJAXU.

https://www.w3schools.com/js/tryit.asp?filename=tryjs_ajax_first

```

<!DOCTYPE html>
<html>
<body>

<div id="demo">
<h1>The XMLHttpRequest Object</h1>
<button type="button" onclick="loadDoc()">Change Content</button>
</div>

<script>
function loadDoc() {
    var xhttp = new XMLHttpRequest();
    xhttp.onreadystatechange = function() {
        if (this.readyState == 4 && this.status == 200) {
            document.getElementById("demo").innerHTML =
                this.responseText;
        }
    };
    xhttp.open("GET", "ajax_info.txt", true);
    xhttp.send();
}
</script>

</body>
</html>

```

The XMLHttpRequest Object

Change Content

AJAX

AJAX is not a programming language.

AJAX is a technique for accessing web servers from a web page.

AJAX stands for Asynchronous JavaScript And XML.

Textově

```
<!DOCTYPE html>
<html>
<body>

<div id="demo">
<h1>The XMLHttpRequest Object</h1>
<button type="button" onclick="loadDoc()">Change
Content</button>
</div>

<script>
function loadDoc() {
    var xhttp = new XMLHttpRequest();
    xhttp.onreadystatechange = function() {
        if (this.readyState == 4 && this.status == 200) {
            document.getElementById("demo").innerHTML =
                this.responseText;
        }
    };
    xhttp.open("GET", "ajax_info.txt", true);
    xhttp.send();
}
</script>

</body>
</html>
```

V této funkci zjistíme, zda **httpRequest.readyState** je roven **4**, což znamená, že je akce hotova.

httpRequest.status je roven **200**, což je status, který vrací server - 200,

pokud je vše OK nebo třeba 404, pokud stránka chybí.

Nejpoužívanější **stavové kódy** jsou následující:

Kód	Popis
200 (OK)	Zdroj byl nalezen a vše je OK
304 (NOT MODIFIED)	Zdroj nebyl od posledního požadavku modifikován. Tento kód je nejčastěji využívají cache webových prohlížečů.
401 (UNAUTHORIZED)	Klient nemá oprávnění pro přístup.
403 (FORBIDDEN)	Klient nezískal oprávnění pro přístup.
404 (NOT FOUND)	Zdroj nebyl nalezen.

7 Čtení souboru XML pomocí asynchronního požadavku XHR

1.1 XML

Použití XML spolu s Ajaxem

Dokument XML by mohl vypadat nějak následovně:

```
<?xml version="1.0" encoding="UTF-8"?>
<CATALOG>
  <CD>
    <TITLE>Empire Burlesque</TITLE>
    <ARTIST>Bob Dylan</ARTIST>
    <COUNTRY>USA</COUNTRY>
    <COMPANY>Columbia</COMPANY>
    <PRICE>10.90</PRICE>
    <YEAR>1985</YEAR>
  </CD>
  <CD>
    <TITLE>Hide your heart</TITLE>
    <ARTIST>Bonnie Tyler</ARTIST>
    <COUNTRY>UK</COUNTRY>
    <COMPANY>CBS Records</COMPANY>
    <PRICE>9.90</PRICE>
    <YEAR>1988</YEAR>
  </CD>
  ...
</CATALOG>
```

https://www.w3schools.com/js/tryit.asp?filename=tryjs_ajax_xml2

The XMLHttpRequest Object

Get my CD collection

Artist	Title
Bob Dylan	Empire Burlesque
Bonnie Tyler	Hide your heart
Dolly Parton	Greatest Hits
Gary Moore	Still got the blues
Eros Ramazzotti	Eros
Bee Gees	One night only
Dr.Hook	Sylvias Mother
Rod Stewart	Maggie May
Andrea Bocelli	Romanza
Percy Sledge	When a man loves a woman
Savage Rose	Black angel
Many	1999 Grammy Nominees
Kenny Rogers	For the good times
Will Smith	Big Willie style
Van Morrison	Tupelo Honey

```

<!DOCTYPE html>
<html>
<style>
table,th,td {
    border : 1px solid black;
    border-collapse: collapse;
}
th,td {
    padding: 5px;
}
</style>
<body>

<h1>The XMLHttpRequest Object</h1>

<button type="button" onclick="loadDoc()">Get my CD collection</button>
<br><br>
<table id="demo"></table>

<script>
function loadDoc() {
    var xhttp = new XMLHttpRequest();
    xhttp.onreadystatechange = function() {
        if (this.readyState == 4 && this.status == 200) {
            myFunction(this);
        }
    };
    xhttp.open("GET", "cd_catalog.xml", true);

```

```

    xhttp.send();
}
function myFunction(xml) {
    var i;
    var xmlDoc = xml.responseXML;
    var table="<tr><th>Artist</th><th>Title</th></tr>";
    var x = xmlDoc.getElementsByTagName("CD");
    for (i = 0; i <x.length; i++) {
        table += "<tr><td>" +
            x[i].getElementsByTagName("ARTIST")[0].childNodes[0].nodeValue +
            "</td><td>" +
            x[i].getElementsByTagName("TITLE")[0].childNodes[0].nodeValue +
            "</td></tr>";
    }
    document.getElementById("demo").innerHTML = table;
}
</script>

</body>
</html>

```

Poznámka

Metoda `getElementsByTagName()` vrátí kolekce všech prvků v dokumentu se zadaným názvem značky, jako `NodeList` objektu.

Objekt **`NodeList`** představuje soubor uzlů. Uzly lze přistupovat pomocí indexů. Index začíná na 0

8 Změna obsahu dokumentu HTML pomocí `innerHTML`

Každý element, jehož obsah chceme změnit, musí mít svoje ID, podle nějž element jednoznačně identifikujeme a můžeme s ním pracovat.

```

<p id='text1'>text1</p>
<div id='text_div'></div>

```

Kdy použít `DIV` a kdy `SPAN`?

Řádkový element `SPAN` v sobě nemůže obsahovat blokové elementy.

Pro výpis bloku textu použít `DIV`.

`DIV` by se zase neměl vyskytovat v řádkovém elementu.

Reference na uzel je pak:

```
var text1 =document.getElementById('text1');
```

9 Ajax - Dynamické vložení tabulky z externího dokumentu (HTML)

Použití: data z databázových tabulek, nákupní košík,...

Tabulka ze souboru tabulka.txt

```
<table border="1">
  <tbody>
    <tr>
      <th>Prezident</th>
      <th>Od</th>
      <th>Do</th>
    </tr>
    <tr>
      <td>George Washington</td>
      <td>1789</td>
      <td>1797</td>
    </tr>
    <tr>
      <td>John Adams</td>
      <td>1797</td>
      <td>1801</td>
    </tr>
    <tr>
      <td>Thomas Jefferson</td>
      <td>1801</td>
      <td>1809</td>
    </tr>
  </tbody>
</table>
```

Poznámka

Chcete-li zpracovat IE5 a IE6, zkontrolujte, zda prohlížeč XMLHttpRequest objekt podporuje , nebo vytvořte ActiveX objekt:

Příklad

```
if (window.XMLHttpRequest) {
  // code for modern browsers
  xmlhttp = new XMLHttpRequest();
} else {
  // code for old IE browsers
  xmlhttp = new ActiveXObject("Microsoft.XMLHTTP");
}
```

Zdrojový kód

```

<html>
<head>
<meta http-equiv="content-type" content="text/html;
charset=windows-1250">

<title>Dynamické vložení tabulky</title>

<script type="text/javascript">

    var xHttp;

    if (window.ActiveXObject)
    { xHttp = new ActiveXObject("Microsoft.XMLHTTP");}
    else if (window.XMLHttpRequest)
    { xHttp = new XMLHttpRequest();}

    function aktivujRequest()
    {
        xHttp.onreadystatechange = obsluhaStateChange;
        xHttp.open("GET", "tabulka.txt", true);
        xHttp.send(null);
    }

    function obsluhaStateChange()
    {
        if(xHttp.readyState == 4 && xHttp.status == 200)
        {
            var nText =
document.getElementById("vlozeni_obsahu");
            nText.innerHTML = xHttp.responseText;
        }
    }

</script>
</head>

<body>
<form action="#">
<input type="button" value="Zobraz tabulku"
onclick="aktivujRequest();"/>
</form>
<div id="vlozeni_obsahu"></div>
</body>
</html>

```

<http://195.113.207.163/~smrcka/wt2/pr3/tabulka.html>

Zobraz tabulku

Prezident	Od	Do
George Washington	1789	1797
John Adams	1797	1801
Thomas Jefferson	1801	1809

10 Vložení textu vygenerovaného serverem v PHP

http://195.113.207.163/~smrcka/wt2/pr3/02/text_ze_serveru.html

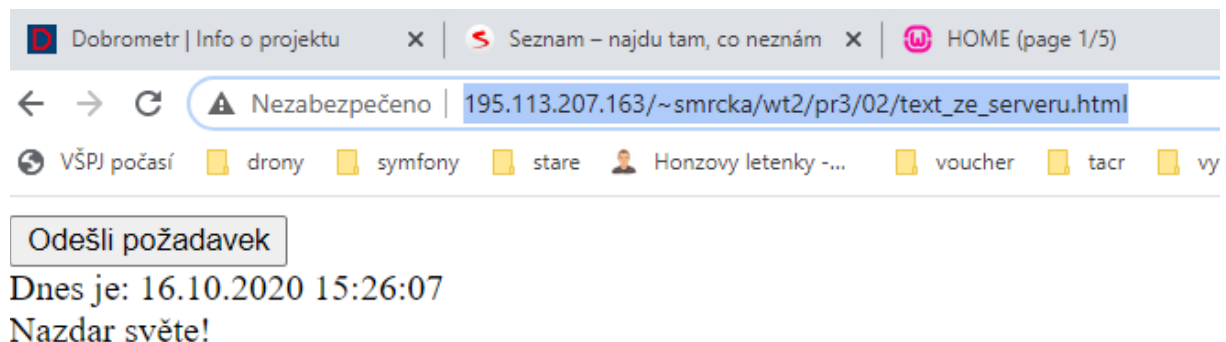
text.php

```
<?php
    echo "Dnes je: ". date("d.m.Y H:i:s")."<br/>";
    echo "Nazdar světe!<br /><br />";
?>
<html>
<head>
<meta http-equiv="content-type" content="text/html;
charset=windows-1250">
<title>Vložení textu ze serveru</title>
<script type="text/javascript">
    var xHttp;
    xHttp = new XMLHttpRequest();
    function aktivujRequest()
    {
        xHttp.onreadystatechange = obsluhaStateChange;
        xHttp.open("GET", "text.php", true);
        xHttp.send(null);
    }
    function obsluhaStateChange()
    {
```

```

        if(xHttp.readyState == 4 && xHttp.status == 200)
        {
            var nText =
document.getElementById("vlozeni_obsahu");
            nText.innerHTML = xHttp.responseText;
        }
    }
</script>
</head>
<body>
<form action="#">
<input type="button" value="Odešli požadavek"
onclick="aktivujRequest();" />
<div id="vlozeni_obsahu"></div>
</form>
</body>
</html>

```



11 Poslání POST požadavku

https://www.w3schools.com/js/tryit.asp?filename=tryjs_ajax_post

```

<html>
<head>
<meta http-equiv="content-type" content="text/html;
charset=windows-1250">
<title>Vložení textu ze serveru</title>
<script type="text/javascript">

var xHttp;
xHttp = new XMLHttpRequest();
function aktivujRequest()
{
    xHttp.onreadystatechange = obsluhaStateChange;

```

```

    xhttp.open("GET", "text.php", true);
    xhttp.send(null);
}
function obsluhaStateChange()
{
    if(xhttp.readyState == 4 && xhttp.status == 200)
    {
        var nText = document.getElementById("vlozeni_obsahu");
        nText.innerHTML = xhttp.responseText;
    }
}
</script>
</head>
<body>
<form action="#">
<input type="button" value="Odešli požadavek" onclick="aktivujRequest();"/>
<div id="vlozeni_obsahu"></div>
</form>
</body>
</html>
<!DOCTYPE html>
<html>
<body>

```

<h2>The XMLHttpRequest Object</h2>

<button type="button" onclick="loadDoc()">Request data</button>

<p id="demo"></p>

```

<script>
function loadDoc() {
    var xhttp = new XMLHttpRequest();
    xhttp.onreadystatechange = function() {
        if (this.readyState == 4 && this.status == 200) {
            document.getElementById("demo").innerHTML = this.responseText;
        }
    };
    xhttp.open("POST", "demo_post.asp", true);
    xhttp.send();
}
</script>

</body>
</html>

```

chcete-li POST data jako formulář HTML, přidejte hlavičku HTTP s `setRequestHeader()`. V `send()` metodě zadejte data, která chcete odeslat :

Příklad

```
xhttp.open("POST", "ajax_test.asp", true);
xhttp.setRequestHeader("Content-type", "application/x-www-form-urlencoded");
xhttp.send("fname=Henry&lname=Ford");
```

Příklad

<body>

<h2>The XMLHttpRequest Object</h2>

<button type="button" onclick="loadDoc()">Request data</button>

<p id="demo"></p>

<script>

```
function loadDoc() {
  var xhttp = new XMLHttpRequest();
  xhttp.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
      document.getElementById("demo").innerHTML = this.responseText;
    }
  };
  xhttp.open("POST", "demo_post2.asp", true);
  xhttp.setRequestHeader("Content-type", "application/x-www-form-urlencoded");
  xhttp.send("fname=Henry&lname=Ford");
}
```

</script>

</body>

</html>

12 PHP s parametrem

https://www.w3schools.com/js/js_ajax_php.asp

Soubor PHP

<?php

// Array with names

\$a[] = "Anna";

\$a[] = "Brittany";

\$a[] = "Cinderella";

\$a[] = "Diana";


```
$a[] = "Eva";  
$a[] = "Fiona";  
$a[] = "Gunda";  
$a[] = "Hege";  
$a[] = "Inga";  
$a[] = "Johanna";  
$a[] = "Kitty";  
$a[] = "Linda";  
$a[] = "Nina";  
$a[] = "Ophelia";  
$a[] = "Petunia";  
$a[] = "Amanda";  
$a[] = "Raquel";  
$a[] = "Cindy";  
$a[] = "Doris";  
$a[] = "Eve";  
$a[] = "Evita";  
$a[] = "Sunniva";  
$a[] = "Tove";  
$a[] = "Unni";  
$a[] = "Violet";  
$a[] = "Liza";  
$a[] = "Elizabeth";  
$a[] = "Ellen";  
$a[] = "Wenche";  
$a[] = "Vicky";
```

```
// get the q parameter from URL
```

```
$q = $_REQUEST["q"];
```

```
$hint = "";
```

```
// lookup all hints from array if $q is different from ""
```

```
if ($q != "") {
```

```
    $q = strtolower($q);
```

```
    $len=strlen($q);
```

```
    foreach($a as $name) {
```

```
        if (stristr($q, substr($name, 0, $len))) {
```

```
            if ($hint == "") {
```

```
                $hint = $name;
```

```
            } else {
```

```
                $hint .= ", $name";
```

```

    }
  }
}

```

```

// Output "no suggestion" if no hint was found or output correct values
echo $hint === "" ? "no suggestion" : $hint;
?>

```

Nejprve zkontrolujte, zda je vstupní pole prázdné (`str.length == 0`). Pokud je to, zrušte obsah `txtHint` zástupný symbol a odchod z funkce.

Nicméně, pokud je vstupní pole není prázdné, proveďte následující kroky:

- Vytvořit objekt `XMLHttpRequest`
- Vytvoření funkce, které mají být provedeny, pokud odpověď serveru je připraven
- Odeslání požadavku off do souboru PHP (`gethint.php`) na serveru
- Všimněte si, že parametr `q` který zní `gethint.php? Q = "+" str`
- Proměnná `str` je držitelem obsah vstupního pole

```

<html>
<head>
<script>
function showHint(str) {
  if (str.length == 0) {
    document.getElementById("txtHint").innerHTML = "";
    return;
  } else {
    var xmlhttp = new XMLHttpRequest();
    xmlhttp.onreadystatechange = function() {
      if (this.readyState == 4 && this.status == 200) {
        document.getElementById("txtHint").innerHTML = this.responseText;
      }
    };
    xmlhttp.open("GET", "gethint.php?q=" + str, true);
    xmlhttp.send();
  }
}
</script>
</head>

```

```
<body>
```

```
<p><b>Start typing a name in the input field below:</b></p>
```

```
<form>
```

```
First name: <input type="text" onkeyup="showHint(this.value)">
```

```
</form>
```

```
<p>Suggestions: <span id="txtHint"></span></p>
```

```
</body>
```

```
</html>
```

The XMLHttpRequest Object

Start typing a name in the input field below:

First name:

Suggestions:

Výsledek po zadání: a

The XMLHttpRequest Object

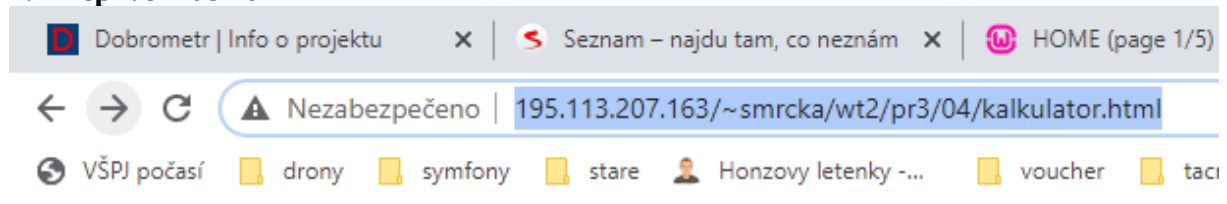
Start typing a name in the input field below:

First name:

Suggestions: Brittany

13 Jednoduchý kalkulátor

1.2 Neprve klasika

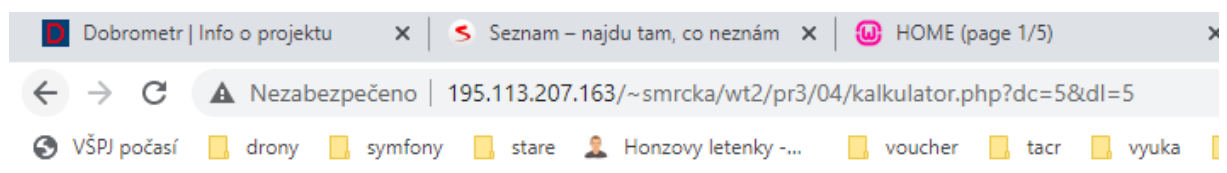


Kalkulátor umožňující dělení

Zadejte dělenec a dělitel:

/

Výsledek



Dělenec: 5

Dělitel: 5

=====

Podíl: 1

<http://195.113.207.163/~smrcka/wt2/pr3/04/kalkulator.html>

```
<html>
<head>
<meta http-equiv="content-type" content="text/html;
  charset=windows-1250">
<title>Kalkulátor umožňující dělení</title>
</head>
<body>
  <h2>Kalkulátor umožňující dělení</h2>
  Zadejte dělenec a dělitel:
  <form name="Kalulator" action="kalkulator.php" method="GET">
    <input type="text" name="dc" value="" />
    <BR>/<BR>
    <input type="text" name="dl" value="" />
    <input type="submit" value="Vydel" />
```

```
</form>
</body>
</html>
```

A php je

```
<?php
    $nDelenec = $_REQUEST["dc"];
    $nDelitel = $_REQUEST["dl"];

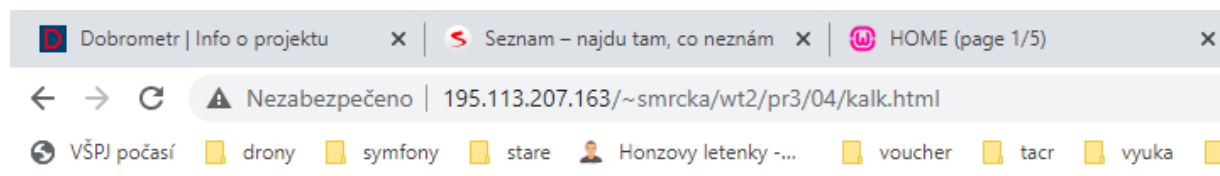
    print "Dělenec: ".$nDelenec."<br/>";
    print "Dělitel: ".$nDelitel."<br/>";
    print "=====<br/>";
    print "Podíl: ".$nDelenec / $nDelitel;
?>
```

Simulovat toto jde:

<http://195.113.207.163/~smrcka/wt2/pr3/04/kalkulator.php?dc=10&dl=5>

14 Interaktivní ajaxový kalkulátor bez potvrzování

<http://195.113.207.163/~smrcka/wt2/pr3/04/kalk.html>



AJAX kalkulátor

Zadejte dělenec a dělitel:

:

Dělenec: 5

Dělitel: 1

=====

Podíl: 5

Php zůstane nezměněno z předchozího příkladu.

```

<head>
<meta http-equiv="content-type" content="text/html;
  charset=windows-1250">

<title>AJAX kalkulátor umožňující dělení </title>
  <script language = "javascript">

    var xHttp;

    function Kalkulator(dataSource)
    {
      xHttp = VytvorXMLHttp();
      xHttp.open("GET", dataSource);
      xHttp.onreadystatechange = function()
      {
        if (xHttp.readyState == 4 && xHttp.status == 200)
        {
          var oznam = document.getElementById("oznam");
          oznam.innerHTML = xHttp.responseText;//získá od serveru
                                //data jako string

        }
      }
      xHttp.send(null); //odeslání požadavku na server
    }

    function DekodujParametr(udalost)
    {
      var dc = document.getElementById("txDelenec");
      var dl = document.getElementById("txDelitel");
      var oznam = document.getElementById("oznam");
      oznam.innerHTML = "<div></div>";
      Kalkulator ("kalkulator.php?dc=" + dc.value + "&dl=" +
dl.value);
    }

    function VytvorXMLHttp()
    {
      var xmlHttp= false;

      if (window.ActiveXObject)
        {xmlHttp = new ActiveXObject("Microsoft.XMLHTTP");}
      else if (window.XMLHttpRequest)
        {xmlHttp = new XMLHttpRequest();}
      return xmlHttp;
    }

  </script>

```

```

</head>
<body>
  <h1>AJAX kalkulátor</h1>
  Zadejte dělenec a dělitel:<br/>
  <input type="text" id = "txDelenec" value = "1"
    onkeyup = "DekodujParametr(event)"/>
  <br/>:<br/>
  <input type="text" id = "txDelitel" value = "1"
    onkeyup = "DekodujParametr(event)"/>
  <div id = "oznam"><div>
</body>
</html>

```

15 Přenos XML

Vlastnost responseXML

Objekt XMLHttpRequest má zabudovaný analyzátor XML.

responseXML Vlastnost vrací odpověď serveru jako objekt XML DOM.

Pomocí této vlastnosti můžete analyzovat odpověď jako objekt XML DOM:

Zdroj

https://www.w3schools.com/js/tryit.asp?filename=tryjs_ajax_responsexml

cd_catalog.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<CATALOG>
  <CD>
    <TITLE>Empire Burlesque</TITLE>
    <ARTIST>Bob Dylan</ARTIST>
    <COUNTRY>USA</COUNTRY>
    <COMPANY>Columbia</COMPANY>
    <PRICE>10.90</PRICE>
    <YEAR>1985</YEAR>
  </CD>
  <CD>
    <TITLE>Hide your heart</TITLE>
    <ARTIST>Bonnie Tyler</ARTIST>
    <COUNTRY>UK</COUNTRY>
    <COMPANY>CBS Records</COMPANY>
    <PRICE>9.90</PRICE>
    <YEAR>1988</YEAR>
  </CD>
</CATALOG>

```

tryjs_ajax_responsexml

```

<!DOCTYPE html>
<html>
<body>

```

<h2>The XMLHttpRequest Object</h2>

<p id="demo"></p>

```
<script>
var xhttp, xmlDoc, txt, x, i;
xhttp = new XMLHttpRequest();
xhttp.onreadystatechange = function() {
  if (this.readyState == 4 && this.status == 200) {
    xmlDoc = this.responseXML;
    txt = "";
    x = xmlDoc.getElementsByTagName("ARTIST");
    for (i = 0; i < x.length; i++) {
      txt = txt + x[i].childNodes[0].nodeValue + "<br>";
    }
    document.getElementById("demo").innerHTML = txt;
  }
};
xhttp.open("GET", "cd_catalog.xml", true);
xhttp.send();
</script>
```

</body>

</html>

The XMLHttpRequest Object

Bob Dylan
Bonnie Tyler

16 Metoda getAllResponseHeaders ()

getAllResponseHeaders() Metoda vrátí všechny informace záhlaví z odpovědi serveru.

Zdroj

https://www.w3schools.com/js/tryit.asp?filename=tryjs_ajax_header

<!DOCTYPE html>

<html>

<body>

<h2>The XMLHttpRequest Object</h2>

<p>The getAllResponseHeaders() function returns all the header information of a resource, like length, server-type, content-type, last-modified, etc:</p>

<p id="demo"></p>

```
<script>
var xhttp = new XMLHttpRequest();
xhttp.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
        document.getElementById("demo").innerHTML =
            this.getAllResponseHeaders();
    }
};
xhttp.open("GET", "ajax_info.txt", true);
xhttp.send();
</script>
```

</body>

</html>

Výsledek

The XMLHttpRequest Object

The getAllResponseHeaders() function returns all the header information of a resource, like length, server-type, content-type, last-modified, etc:

accept-ranges: bytes age: 2083 cache-control: public,max-age=14400,public content-encoding: gzip content-length: 245 content-type: text/plain date: Fri, 16 Oct 2020 16:50:11 GMT etag: "15bfdeee0ffd21:0" last-modified: Tue, 18 Jul 2017 16:14:29 GMT server: ECS (fcn/40FB) status: 304 vary: Accept-Encoding x-cache: HIT x-frame-options: SAMEORIGIN x-powered-by: ASP.NET